

```

    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 4.3.4 节例题 14
同类题目：Wall, NEERC 200, HDU 1348
*/

```

本节例题列表

本节讲解的例题及其囊括的知识点，如表6-3所示。

表6-3 二维几何常用算法例题归纳

编 号	题 号	标 题	知 识 点	代 码 作 者
例 6-5	UVa 10652	Board Wrapping	凸包	刘汝佳
例 6-6	UVa 11168	Airport	凸包；直线的一般式	刘汝佳
例 6-7	UVa 1453	Square	凸包；旋转卡壳	刘汝佳
例 6-8	UVa 1396	Most Distant Point from the Sea	二分法；半平面交	刘汝佳
例 6-9	POJ 2048	Monster Trap	平面区域；图论模型	刘汝佳
例 6-10	UVa 1340	Find the Border	平面区域；卷包裹法	刘汝佳

6.4 三维几何基础

本节提供一些与三维几何基础算法相关的经典题目及其代码实现。

例6-11 【线段和三角形相交（3D）】 三维三角形（3D Triangles, UVa 11275）

给出三维空间的两个三角形，判断两者是否有公共点。若两个点距离小于0.000 001，则认为它们是同一个点。三角形的内部、边和顶点都看成三角形的一部分。如果两个三角形有公共点，则输出1，否则输出0。

【代码实现】

```
// 刘汝佳
#include <bits/stdc++.h>
using namespace std;

struct Point3 {
    double x, y, z;
    Point3(double x = 0, double y = 0, double z = 0): x(x), y(y), z(z) { }
};

istream& operator>>(istream& is, Point3& p) { return is >> p.x >> p.y >> p.z; }

typedef Point3 Vector3;

Vector3 operator + (const Vector3& A, const Vector3& B)
{ return Vector3(A.x + B.x, A.y + B.y, A.z + B.z); }
```

```

Vector3 operator - (const Point3& A, const Point3& B)
{ return Vector3(A.x - B.x, A.y - B.y, A.z - B.z); }
Vector3 operator * (const Vector3& A, double p)
{ return Vector3(A.x * p, A.y * p, A.z * p); }
Vector3 operator / (const Vector3& A, double p)
{ return Vector3(A.x / p, A.y / p, A.z / p); }

const double eps = 1e-8;
int dcmp(double x) {
    if (fabs(x) < eps) return 0; else return x < 0 ? -1 : 1;
}

double Dot(const Vector3& A, const Vector3& B)
{ return A.x * B.x + A.y * B.y + A.z * B.z; }
double Length(const Vector3& A) { return sqrt(Dot(A, A)); }
double Angle(const Vector3& A, const Vector3& B)
{ return acos(Dot(A, B) / Length(A) / Length(B)); }
Vector3 Cross(const Vector3& A, const Vector3& B)
{ return Vector3(A.y * B.z - A.z * B.y, A.z * B.x - A.x * B.z, A.x * B.y - A.y * B.x); }
double Area2(const Point3& A, const Point3& B, const Point3& C)
{ return Length(Cross(B - A, C - A)); }

// p1 和 p2 是否在线段 a-b 的同侧
bool SameSide(const Point3& p1, const Point3& p2,
               const Point3& a, const Point3& b) {
    return dcmp(Dot(Cross(b - a, p1 - a), Cross(b - a, p2 - a))) >= 0;
}

// 点在三角形 P0P1P2 中
bool PointInTri(const Point3& P, const Point3& P0,
                const Point3& P1, const Point3& P2) {
    return SameSide(P, P0, P1, P2)
        && SameSide(P, P1, P0, P2) && SameSide(P, P2, P0, P1);
}

// 三角形 P0P1P2 是否和线段 AB 相交
bool TriSegIntersection(const Point3& P0, const Point3& P1, const Point3& P2,
                        const Point3& A, const Point3& B, Point3& P) {
    Vector3 n = Cross(P1 - P0, P2 - P0);
    if (dcmp(Dot(n, B - A)) == 0) return false; // 线段 AB 和平面 P0P1P2 平行或共面
    else { // 平面 A 和直线 P1P2 有唯一交点
        double t = Dot(n, P0 - A) / Dot(n, B - A);
        if (dcmp(t) < 0 || dcmp(t - 1) > 0) return false; // 不在线段 AB 上
        P = A + (B - A) * t; // 交点
        return PointInTri(P, P0, P1, P2);
    }
}

```

```

bool TriTriIntersection(Point3* T1, Point3* T2) {
    Point3 P;
    for (int i = 0; i < 3; i++) {
        if (TriSegIntersection(T1[0], T1[1], T1[2], T2[i], T2[(i + 1) % 3], P))
            return true;
        if (TriSegIntersection(T2[0], T2[1], T2[2], T1[i], T1[(i + 1) % 3], P))
            return true;
    }
    return false;
}

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    int T; cin >> T;
    while (T--) {
        Point3 T1[3], T2[3];
        for (int i = 0; i < 3; i++) cin >> T1[i];
        for (int i = 0; i < 3; i++) cin >> T2[i];
        cout << (TriTriIntersection(T1, T2) ? 1 : 0) << endl;
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 4.4.4 节例题 16
*/

```

例6-12 【线段之间的距离（3D）】 Ardenia王国（Ardenia, CERC 2010, UVa 1469）

给出空间中两条线段，求它们的最近距离。输出两个互素的整数 l 和 m ，使得 $m > 0$ 且 l/m 为所求距离的平方。

【代码实现】

```

// 刘汝佳
#include <bits/stdc++.h>
using namespace std;

struct Point3 {
    int x, y, z;
    Point3(int x = 0, int y = 0, int z = 0) : x(x), y(y), z(z) {}
};

istream& operator>>(istream& is, Point3& p) { return is >> p.x >> p.y >> p.z; }

typedef Point3 Vector3;

Vector3 operator+(const Vector3& A, const Vector3& B)
{ return Vector3(A.x + B.x, A.y + B.y, A.z + B.z);}
Vector3 operator-(const Point3& A, const Point3& B)
{ return Vector3(A.x - B.x, A.y - B.y, A.z - B.z);}
Vector3 operator*(const Vector3& A, int p)
{ return Vector3(A.x * p, A.y * p, A.z * p);}

```

```

bool operator==(const Point3& a, const Point3& b)
{ return a.x == b.x && a.y == b.y && a.z == b.z; }
int Dot(const Vector3& A, const Vector3& B)
{ return A.x * B.x + A.y * B.y + A.z * B.z; }
int Length2(const Vector3& A) { return Dot(A, A); }
Vector3 Cross(const Vector3& A, const Vector3& B) {
    return Vector3(A.y * B.z - A.z * B.y, A.z * B.x - A.x * B.z,
        A.x * B.y - A.y * B.x);
}

typedef long long LL;
LL gcd(LL a, LL b) { return b ? gcd(b, a % b) : a; }
LL lcm(LL a, LL b) { return a / gcd(a, b) * b; }
struct Rat {
    LL a, b;
    Rat(LL a = 0) : a(a), b(1) {}
    Rat(LL x, LL y) : a(x), b(y) {
        if (b < 0) a = -a, b = -b;
        LL d = gcd(a, b);
        if (d < 0) d = -d;
        a /= d, b /= d;
    }
};

Rat operator+(const Rat& A, const Rat& B) {
    LL x = lcm(A.b, B.b);
    return Rat(A.a * (x / A.b) + B.a * (x / B.b), x);
}
Rat operator-(const Rat& A, const Rat& B) { return A + Rat(-B.a, B.b); }
Rat operator*(const Rat& A, const Rat& B) { return Rat(A.a * B.a, A.b * B.b); }
void updatemin(Rat& A, const Rat& B) {
    if (A.a * B.b > B.a * A.b) A.a = B.a, A.b = B.b;
}

// 点 P 到线段 AB 的距离的平方
Rat Rat_Dist2ToSeg(const Point3& P, const Point3& A, const Point3& B) {
    if (A == B) return Length2(P - A);
    Vector3 v1 = B - A, v2 = P - A, v3 = P - B;
    if (Dot(v1, v2) < 0) return Length2(v2);
    if (Dot(v1, v3) > 0) return Length2(v3);
    return Rat(Length2(Cross(v1, v2)), Length2(v1));
}

// 求异面直线 p1+su 和 p2+tv 的公垂线对应的 s。如果平行/重合, 返回 false
bool Rat_LineDistance3D(const Point3& p1, const Vector3& u, const Point3& p2,
    const Vector3& v, Rat& s) {
    LL b = (LL)Dot(u, u) * Dot(v, v) - (LL)Dot(u, v) * Dot(u, v);
    if (b == 0) return false;
    LL a = (LL)Dot(u, v) * Dot(v, p1 - p2) - (LL)Dot(v, v) * Dot(u, p1 - p2);
    s = Rat(a, b);
}

```

```

    return true;
}

void Rat_GetPointOnLine(const Point3& A, const Point3& B, const Rat& t, Rat& x,
                       Rat& y, Rat& z) {
    x = Rat(A.x) + Rat(B.x - A.x) * t;
    y = Rat(A.y) + Rat(B.y - A.y) * t;
    z = Rat(A.z) + Rat(B.z - A.z) * t;
}

Rat Rat_Distance2(const Rat& x1, const Rat& y1, const Rat& z1, const Rat& x2,
                  const Rat& y2, const Rat& z2) {
    return (x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2)
        + (z1 - z2) * (z1 - z2);
}

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    int T;
    cin >> T;
    for (Point3 A, B, C, D; cin >> A >> B >> C >> D, T--;) {
        Rat s, t, ans = Rat(1e9);
        bool ok = false;
        if (Rat_LineDistance3D(A, B - A, C, D - C, s))
            if (s.a > 0 && s.a < s.b && Rat_LineDistance3D(C, D - C, A, B - A, t))
                if (t.a > 0 && t.a < t.b) {
                    ok = true; // 异面直线/相交直线
                    Rat x1, y1, z1, x2, y2, z2;
                    Rat_GetPointOnLine(A, B, s, x1, y1, z1),
                        Rat_GetPointOnLine(C, D, t, x2, y2, z2);
                    ans = Rat_Distance2(x1, y1, z1, x2, y2, z2);
                }
        if (!ok) { // 平行直线/重合直线
            updatemin(ans, Rat_Dist2ToSeg(A, C, D)),
                updatemin(ans, Rat_Dist2ToSeg(B, C, D));
            updatemin(ans, Rat_Dist2ToSeg(C, A, B)),
                updatemin(ans, Rat_Dist2ToSeg(D, A, B));
        }
        cout << ans.a << " " << ans.b << endl;
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 4.4.4 节例题 17
同类题目：The Deadly Olympic Returns, UVa 10794
*/

```

例6-13 【三维凸包；点到平面的距离】行星 (Asteroids, NEERC 2009, UVa 1438)

给定两颗凸多面体行星，你的任务是求出二者重心的最近距离。两颗行星的密度是均匀分布的，且可以任意旋转和平移。每颗行星顶点数不超过60。输入包含多组数据，每组数据分成两部分，依次描述两颗行星。每颗行星第一行为顶点数 n ($4 \leq n \leq 60$)，以下 n 行每行给出一个顶点坐标。这 n 个点保证是一个非退化凸多面体的各个顶点。

对于每组数据，输出两个行星重心的最短距离。

【代码实现】


```

// 刘汝佳
#include <vector>
#include <cmath>
#include <cstdio>
#include <cstdlib>
#include <iostream>
using namespace std;

const double eps = 1e-8;
int dcmp(double x) {
    if (fabs(x) < eps) return 0;
    return x < 0 ? -1 : 1;
}

struct Point3 {
    double x, y, z;
    Point3(double x = 0, double y = 0, double z = 0): x(x), y(y), z(z) { }
};

istream& operator>>(istream& is, Point3& p) { return is >> p.x >> p.y >> p.z; }

typedef Point3 Vector3;

Vector3 operator + (const Vector3& A, const Vector3& B)
{ return Vector3(A.x + B.x, A.y + B.y, A.z + B.z); }
Vector3 operator - (const Point3& A, const Point3& B)
{ return Vector3(A.x - B.x, A.y - B.y, A.z - B.z); }
Vector3 operator * (const Vector3& A, double p)
{ return Vector3(A.x * p, A.y * p, A.z * p); }
Vector3 operator / (const Vector3& A, double p)
{ return Vector3(A.x / p, A.y / p, A.z / p); }
bool operator == (const Point3& a, const Point3& b)
{ return dcmp(a.x - b.x) == 0 && dcmp(a.y - b.y) == 0 && dcmp(a.z - b.z) == 0; }
double Dot(const Vector3& A, const Vector3& B) { return A.x * B.x + A.y * B.y
+ A.z * B.z; }
double Length(const Vector3& A) { return sqrt(Dot(A, A)); }
double Angle(const Vector3& A, const Vector3& B)
{ return acos(Dot(A, B) / Length(A) / Length(B)); }
Vector3 Cross(const Vector3& A, const Vector3& B)
{ return Vector3(A.y*B.z - A.z*B.y, A.z*B.x - A.x*B.z, A.x*B.y - A.y*B.x); }
double Area2(const Point3& A, const Point3& B, const Point3& C)
{ return Length(Cross(B - A, C - A)); }
double Volume6(const Point3& A, const Point3& B, const Point3& C, const Point3& D)

```

```

{ return Dot(D - A, Cross(B - A, C - A)); }
Point3 Centroid(const Point3& A, const Point3& B, const Point3& C, const Point3& D)
{ return (A + B + C + D) / 4.0; }
double rand01() { return rand() / (double)RAND_MAX; }
double randeps() { return (rand01() - 0.5) * eps; }
Point3 add_noise(const Point3& p)
{ return Point3(p.x + randeps(), p.y + randeps(), p.z + randeps()); }

struct Face {
    int v[3];
    Face(int a, int b, int c) { v[0] = a; v[1] = b; v[2] = c; }
    Vector3 Normal(const vector<Point3>& P) const {
        return Cross(P[v[1]] - P[v[0]], P[v[2]] - P[v[0]]);
    }
    // f 是否能看见 P[i]
    int CanSee(const vector<Point3>& P, int i) const {
        return Dot(P[i] - P[v[0]], Normal(P)) > 0;
    }
};

// 增量法求三维凸包
// 注意：这里没有考虑各种特殊情况（如四点共面）。实践中需在调用前对输入点进行微小扰动
vector<Face> CH3D(const vector<Point3>& P) {
    int n = P.size();
    vector<vector<int>> > vis(n);
    for (int i = 0; i < n; i++) vis[i].resize(n);

    vector<Face> cur;
    cur.push_back(Face(0, 1, 2)); // 由于已经进行扰动，前 3 个点不共线
    cur.push_back(Face(2, 1, 0));
    for (int i = 3; i < n; i++) {
        vector<Face> next;
        // 计算每条边的“左面”的可见性
        for (size_t j = 0; j < cur.size(); j++) {
            Face& f = cur[j];
            int res = f.CanSee(P, i);
            if (!res) next.push_back(f);
            for (int k = 0; k < 3; k++) vis[f.v[k]][f.v[(k + 1) % 3]] = res;
        }
        for (size_t j = 0; j < cur.size(); j++)
            for (int k = 0; k < 3; k++) {
                int a = cur[j].v[k], b = cur[j].v[(k + 1) % 3];
                if (vis[a][b] != vis[b][a] && vis[a][b]) // (a,b) 是分界线，左边对 P[i] 可见
                    next.push_back(Face(a, b, i));
            }
        cur = next;
    }
    return cur;
}

```

```

struct ConvexPolyhedron {
    int n;
    vector<Point3> P, P2;
    vector<Face> faces;

    bool read() {
        if (!(cin >> n)) return false;
        P.resize(n), P2.resize(n);
        for (int i = 0; i < n; i++) cin >> P[i], P2[i] = add_noise(P[i]);
        faces = CH3D(P2);
        return true;
    }

    Point3 centroid() {
        Point3 C = P[0];
        double totv = 0;
        Point3 tot(0, 0, 0);
        for (size_t i = 0; i < faces.size(); i++) {
            Point3 p1 = P[faces[i].v[0]], p2 = P[faces[i].v[1]], p3 = P[faces[i].v[2]];
            double v = -Volume6(p1, p2, p3, C);
            totv += v;
            tot = tot + Centroid(p1, p2, p3, C) * v;
        }
        return tot / totv;
    }

    double mindist(Point3 C) {
        double ans = 1e30;
        for (size_t i = 0; i < faces.size(); i++) {
            Point3 p1 = P[faces[i].v[0]], p2 = P[faces[i].v[1]], p3 = P[faces[i].v[2]];
            ans = min(ans, fabs(-Volume6(p1, p2, p3, C) / Area2(p1, p2, p3)));
        }
        return ans;
    }
};

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    ConvexPolyhedron P1, P2;
    while (P1.read() && P2.read()) {
        Point3 C1 = P1.centroid();
        double d1 = P1.mindist(C1);
        Point3 C2 = P2.centroid();
        double d2 = P2.mindist(C2);
        printf("%.8lf\n", d1 + d2);
    }
    return 0;
}

```

/*

算法分析请参考：《算法竞赛入门经典——训练指南》升级版 4.4.4 节例题 18

同类题目：Star War, UVa 11836

*/

例6-14 【三维几何综合】压纸器 (Paperweight, World Finals 2010, UVa 1100)

你的公司从事压纸器的生产，压纸器由两个有公共面的四面体组成，四面体由透明玻璃构成，内部镶嵌数个彩色微粒，其中有一个微粒是一个RFID芯片。

使用时，需要把压纸器稳定地放置在一台计算机的顶部，其中紧贴计算机的那个面称为压纸器的底面。所谓“稳定”，是指压纸器的重心向任意方向移动不超过0.2米时，压纸器都能回到原位。你可以认为压纸器密度均匀，并且芯片体积足够小（可以被认为是一个点）。你的任务是计算压纸器处于稳定位置时RFID芯片离底面的最短距离和最长距离（见图6-8）。

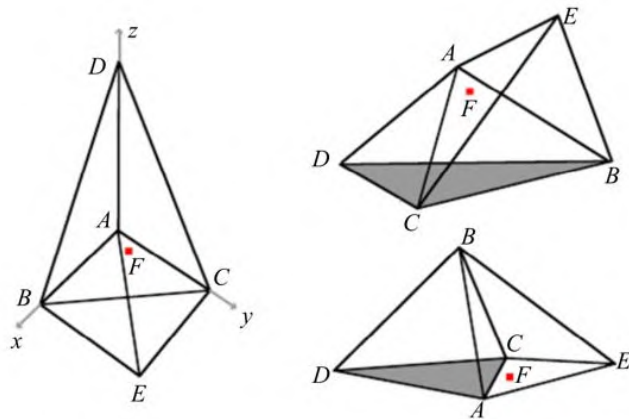


图6-8 压纸器

【代码实现】

```

// 刘汝佳
#include <cstdio>
#include <cmath>
using namespace std;

const double eps = 1e-8;
int dcmp(double x) {
    if(fabs(x) < eps) return 0; else return x < 0 ? -1 : 1;
}

struct Point3 {
    double x, y, z;
    Point3(double x=0, double y=0, double z=0):x(x),y(y),z(z) { }
};

typedef Point3 Vector3;

Vector3 operator + (const Vector3& A, const Vector3& B) {
    return Vector3(A.x+B.x, A.y+B.y, A.z+B.z);
}
Vector3 operator - (const Point3& A, const Point3& B) {
    return Vector3(A.x-B.x, A.y-B.y, A.z-B.z);
}
Vector3 operator * (const Vector3& A, double p) {
    return Vector3(A.x*p, A.y*p, A.z*p);
}
Vector3 operator / (const Vector3& A, double p) {

```

```

    return Vector3(A.x/p, A.y/p, A.z/p);
}
double Dot(const Vector3& A, const Vector3& B)
{ return A.x*B.x + A.y*B.y + A.z*B.z; }
double Length(const Vector3& A) { return sqrt(Dot(A, A)); }
double Angle(const Vector3& A, const Vector3& B)
{ return acos(Dot(A, B) / Length(A) / Length(B)); }
Vector3 Cross(const Vector3& A, const Vector3& B)
{ return Vector3(A.y*B.z - A.z*B.y, A.z*B.x - A.x*B.z, A.x*B.y - A.y*B.x); }
double Area2(const Point3& A, const Point3& B, const Point3& C)
{ return Length(Cross(B-A, C-A)); }
double Volume6(const Point3& A, const Point3& B,
    const Point3& C, const Point3& D)
{ return Dot(D-A, Cross(B-A, C-A)); }
bool read_point3(Point3& p) {
    if(scanf("%lf%lf%lf", &p.x, &p.y, &p.z) != 3) return false;
    return true;
}
// 点p到平面p0-n的距离, n必须为单位向量
double DistanceToPlane(const Point3& p, const Point3& p0, const Vector3& n) {
    return fabs(Dot(p-p0, n)); // 如果不取绝对值, 得到的是有向距离
}
// 点p在平面p0-n上的投影, n必须为单位向量
Point3 GetPlaneProjection(const Point3& p, const Point3& p0, const Vector3& n){
    return p-n*Dot(p-p0, n);
}

// 点P到直线AB的距离
double DistanceToLine(const Point3& P, const Point3& A, const Point3& B) {
    Vector3 v1 = B - A, v2 = P - A;
    return Length(Cross(v1, v2)) / Length(v1);
}
// p1和p2是否在线段ab的同侧
bool SameSide(const Point3& p1, const Point3& p2,
    const Point3& a, const Point3& b) {
    return dcmp(Dot(Cross(b-a, p1-a), Cross(b-a, p2-a))) >= 0;
}

// 点在三角形P0P1P2中
bool PointInTri(const Point3& P, const Point3& P0,
    const Point3& P1, const Point3& P2) {
    return SameSide(P, P0, P1, P2) && SameSide(P, P1, P0, P2)
        && SameSide(P, P2, P0, P1);
}
// 四面体的重心
Point3 Centroid(const Point3& A, const Point3& B,
    const Point3& C, const Point3& D) {
    return (A + B + C + D)/4.0;
}

```

```

#include <algorithm>
using namespace std;

// 判断 P 是否在三角形 ABC 中，并且到 3 条边的距离都至少为 mindist。保证 P, A, B, C 共面
bool InsideWithMinDistance(const Point3& P, const Point3& A, const Point3& B,
const Point3& C, double mindist) {
    if(!PointInTri(P, A, B, C)) return false;
    if(DistanceToLine(P, A, B) < mindist) return false;
    if(DistanceToLine(P, B, C) < mindist) return false;
    if(DistanceToLine(P, C, A) < mindist) return false;
    return true;
}

// 判断 P 是否在凸四边形 ABCD（顺时针或逆时针）中，并且到 4 条边的距离都至少为 mindist。保证
P, A, B, C, D 共面
bool InsideWithMinDistance(const Point3& P, const Point3& A, const Point3& B,
const Point3& C, const Point3& D, double mindist) {
    if(!PointInTri(P, A, B, C)) return false;
    if(!PointInTri(P, C, D, A)) return false;
    if(DistanceToLine(P, A, B) < mindist) return false;
    if(DistanceToLine(P, B, C) < mindist) return false;
    if(DistanceToLine(P, C, D) < mindist) return false;
    if(DistanceToLine(P, D, A) < mindist) return false;
    return true;
}

int main() {
    for(int kase = 1; ; kase++) {
        Point3 P[5], F;
        for(int i = 0; i < 5; i++)
            if(!read_point3(P[i])) return 0;
        read_point3(F);

        // 求重心坐标
        Point3 c1 = Centroid(P[0], P[1], P[2], P[3]);
        Point3 c2 = Centroid(P[0], P[1], P[2], P[4]);
        double vol1 = fabs(Volume6(P[0], P[1], P[2], P[3])) / 6.0;
        double vol2 = fabs(Volume6(P[0], P[1], P[2], P[4])) / 6.0;
        Point3 centroid = (c1 * vol1 + c2 * vol2) / (vol1 + vol2);

        // 枚举放置方案
        double mindist = 1e9, maxdist = -1e9;
        for(int i = 0; i < 5; i++)
            for(int j = i+1; j < 5; j++)
                for(int k = j+1; k < 5; k++) {
                    // 找出另外两个点的下标 a 和 b
                    int vis[5] = {0};
                    vis[i] = vis[j] = vis[k] = 1;
                }
    }
}

```

```

int a, b;
for(a = 0; a < 5; a++) if(!vis[a]) { b = 10-i-j-k-a; break; }

// 判断a和b是否在平面i-j-k的异侧
int d1 = dcmp(Volume6(P[i], P[j], P[k], P[a]));
int d2 = dcmp(Volume6(P[i], P[j], P[k], P[b]));
if(d1 * d2 < 0) continue;           // 是, 则放置方案不合法

Vector3 n = Cross(P[j]-P[i], P[k]-P[i]);    // 法向量
n = n / Length(n);                          // 单位化
// 重心在平面i-j-k上的投影
Point3 proj=GetPlaneProjection(centroid,P[i],n);
bool ok = InsideWithMinDistance(proj, P[i], P[j], P[k], 0.2);
if(!ok) {
    if(d1==0){
        // i, j, k, a 四点共面。i和j一定为三角形ABC 3个顶点之一, k和a是D或E
        if(!InsideWithMinDistance(proj,P[i],P[k],P[j],P[a],0.2))
            continue;
    } else if(d2 == 0){
        // i, j, k, b 四点共面。i和j一定为三角形ABC 3个顶点之一, k和b是D或E
        if(!InsideWithMinDistance(proj,P[i],P[k],P[j],P[b],0.2))
            continue;
    } else
        continue;
}

// 更新答案
double dist = DistanceToPlane(F, P[i], n);
mindist = min(mindist, dist);
maxdist = max(maxdist, dist);
}
printf("Case %d: %.5lf %.5lf\n", kase, mindist, maxdist);
}
return 0;
}
/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 4.4.4 节例题 19
同类题目: Don't Burst the Balloon, ACM/ICPC 会津 2013, Aizu 1342
*/

```

例6-15 【三维几何；立方体切割】黄金屋顶（The Golden Ceiling, ACM/ICPC, Greater NY 2011, HDU 4244）

给出一个长宽高为 L, W, H 的立方体，立方体的一个顶点位于原点，并且其中所有点的坐标值都是正数。给出一个平面 $Ax + By + Cz = D$ ，平面不是垂直的（ $C \geq 1$ ），并且平面一定会穿过立方体内

部。也就是说，一定会有一个点 (x,y,z) 在立方体内部且在平面上方 $(Ax+By+Cz>D)$ ，并且还有其他点在平面下方。这个平面穿过立方体可能有很多种情况，计算切割出来的朝上的表面积，包含顶部剩下的面积，以及切割出来的斜面面积，也就是说，从 z 轴无限远处能看到的立方体面积。图6-9是几种可能的切割情况。

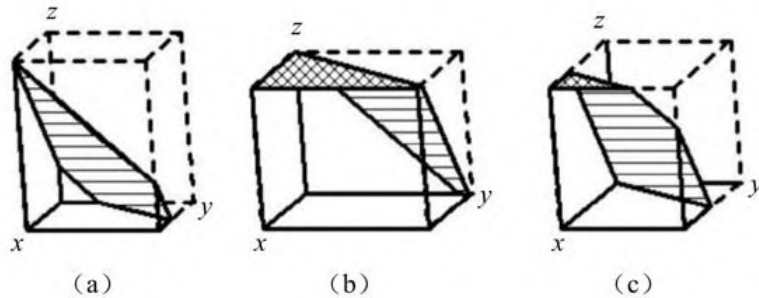


图6-9 立方体切割

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
using namespace std;
typedef long long LL;
const double EPS = 0.0001;
struct Point3 {
    double x, y, z;
    Point3(double _x = 0, double _y = 0, double _z = 0):x(_x), y(_y), z(_z) {}
};
typedef Point3 Vector3;
Vector3 operator + (const Vector3& A, const Vector3& B)
{ return Vector3(A.x + B.x, A.y + B.y, A.z + B.z); }
Vector3 operator - (const Point3& A, const Point3& B)
{ return Vector3(A.x - B.x, A.y - B.y, A.z - B.z); }
Vector3 operator * (const Vector3& A, double p)
{ return Vector3(A.x * p, A.y * p, A.z * p); }
double dot(const Point3& A, const Point3& B)
{ return A.x * B.x + A.y * B.y + A.z * B.z; }
double length(const Point3& A) { return sqrt(dot(A, A)); }
// 线段 p1-p2 和平面  $PV \cdot V = D$  的交点
Point3 LinePlaneIntersection(const Point3& p1, const Point3& p2,
                             const Vector3& PV, double D) {
    double v1 = dot(p1, PV) - D, v2 = dot(p2, PV) - D, denom = v1 - v2;
    return p1 * (-v2 / denom) + p2 * (v1 / denom);
}

// 这些顶点组成的多边形位于平面  $PV \cdot D$  下方的多边形面积
double area_under(const vector<Point3>& ps, const Vector3& PV, double D) {
    vector<Point3> poly;
    for (size_t i = 0; i < ps.size(); i++) {
        const Point3 &p = ps[i], &np = ps[(i + 1) % ps.size()];
        double v = dot(PV, p) - D, nv = dot(PV, np) - D;
        if (v <= EPS) poly.push_back(p); // v 在平面下方或者经过平面
        if (v*nv < EPS && fabs(v)>EPS && fabs(nv)>EPS) // vi 和 nxt_vi 形成的边和平面相交
    }
}

```

```

    poly.push_back(LinePlaneIntersection(np, p, PV, D)); // 边和平面的交点
}
double s = 0; // 多边形都是和 xy 平面平行的
for (size_t j = 1; j + 1 < poly.size(); j++) {
    Point3 v1 = poly[j] - poly[0], v2 = poly[j + 1] - poly[0];
    s += v1.x * v2.y - v2.x * v1.y;
}
return fabs(s) / 2.0;
}

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    Point3 VS[8]; // 8 个顶点
    Vector3 PV; // 平面向量{A, B, C}
    double D, L, W, H;
    int T; cin >> T;
    for (int t = 1, N; t <= T; t++) {
        cin >> N >> L >> W >> H;
        cin >> PV.x >> PV.y >> PV.z >> D;
        for (int i = 0; i < 8; i++) { // 注意立方体顶点坐标
            Point3& p = VS[i];
            if (i & 1) p.z = H;
            if (i & 2) p.y = W;
            if (i & 4) p.x = L;
        }
        vector<Point3> top_vs = {VS[1], VS[3], VS[7], VS[5]},
            bottom_vs = {VS[0], VS[2], VS[6], VS[4]};
        double top_area = all_of(top_vs.begin(), top_vs.end(),
            [&](const Point3 & p) {return dot(PV, p) >= D;})
            ? 0.0 : area_under(top_vs, PV, D); // 平面和顶面是否没有公共点

        double bottom_area = all_of(bottom_vs.begin(), bottom_vs.end(),
            [&](const Point3 & p) {return dot(PV, p) <= D;})
            ? L * W : area_under(bottom_vs, PV, D); // 底面 4 个点是否都在平面 PV-D 下
        double cos_phi = PV.z / length(PV); // 平面法向量和 z 轴夹角的余弦值
        double ans = top_area + (bottom_area - top_area) / cos_phi;
        printf("%d %.01f\n", t, ceil(ans));
    }
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 4.4.4 节例题 20
同类题目: Intersection of Two Prisms, ACM/ICPC 东京 2010, UVa 1483
*/

```

本节例题列表